SIMATS
ENGINEERING

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Hospital Patient Record System with Data Management Using Structures and Pointer-Based Access

CSA0203 – C Programming

Presented by:
V.Vijay(192525264)
A Guna Sekar(192525302)
P V Rahul Kavi(192571090)

Abstract

C-based system for efficient hospital patient record management.

Uses **structures and pointers** for organized data access.

Includes **Patient Registration, Medical History, Treatment, and Billing** .

Supports **CRUD operations** – Create, Read, Update, Delete.

Uses **dynamic memory allocation** for efficient memory management.

Improves **data organization, accuracy, and accessibility**

Introduction

Hospital records contain important **patient and treatment information** .

Manual record management can be **time-consuming and difficult to maintain** .

This project provides a **computerized patient record management system** .

Structures organize different types of patient information.

Pointers provide efficient access to stored records.

The system simplifies **registration, medical history, treatment, and billing** .

Problem Statement

Manual records are **time-consuming** to maintain.

Patient information can be **lost or damaged**.

Difficult to **search and update** patient records.

Repeated data entry may cause **errors and duplication** .

Managing **medical history and billing** manually is inefficient.

Lack of proper data management reduces **accuracy and accessibility** .

Existing System

Patient records are mainly maintained using **manual files and registers** .

Searching patient information is **slow and difficult** .

Updating records requires **repeated manual work** .

Records may be **lost, damaged, or duplicated** .

Medical history and billing information are **difficult to manage efficiently** .

Limited automation leads to **lower accuracy and productivity** .

Proposed System

A **computerized system** for managing hospital patient records.

Uses **structures and pointers** for efficient data handling.

Provides **registration, medical history, treatment, and billing** modules.

Supports **Create, Read, Update, and Delete (CRUD)** operations.

Uses **dynamic memory allocation** for effective memory utilization.

Provides **faster, accurate, and organized** patient record management.

Objectives

To develop an **efficient patient record management system** .

To organize patient data using **C structures** .

To implement **pointer-based data access** .

To perform **CRUD operations** on patient records.

To manage **medical history, treatment, and billing** efficiently.

To improve **data accuracy and accessibility** .

System Functionality

Patient Registration – Add and manage patient details.

Medical History – Store and view previous medical records.

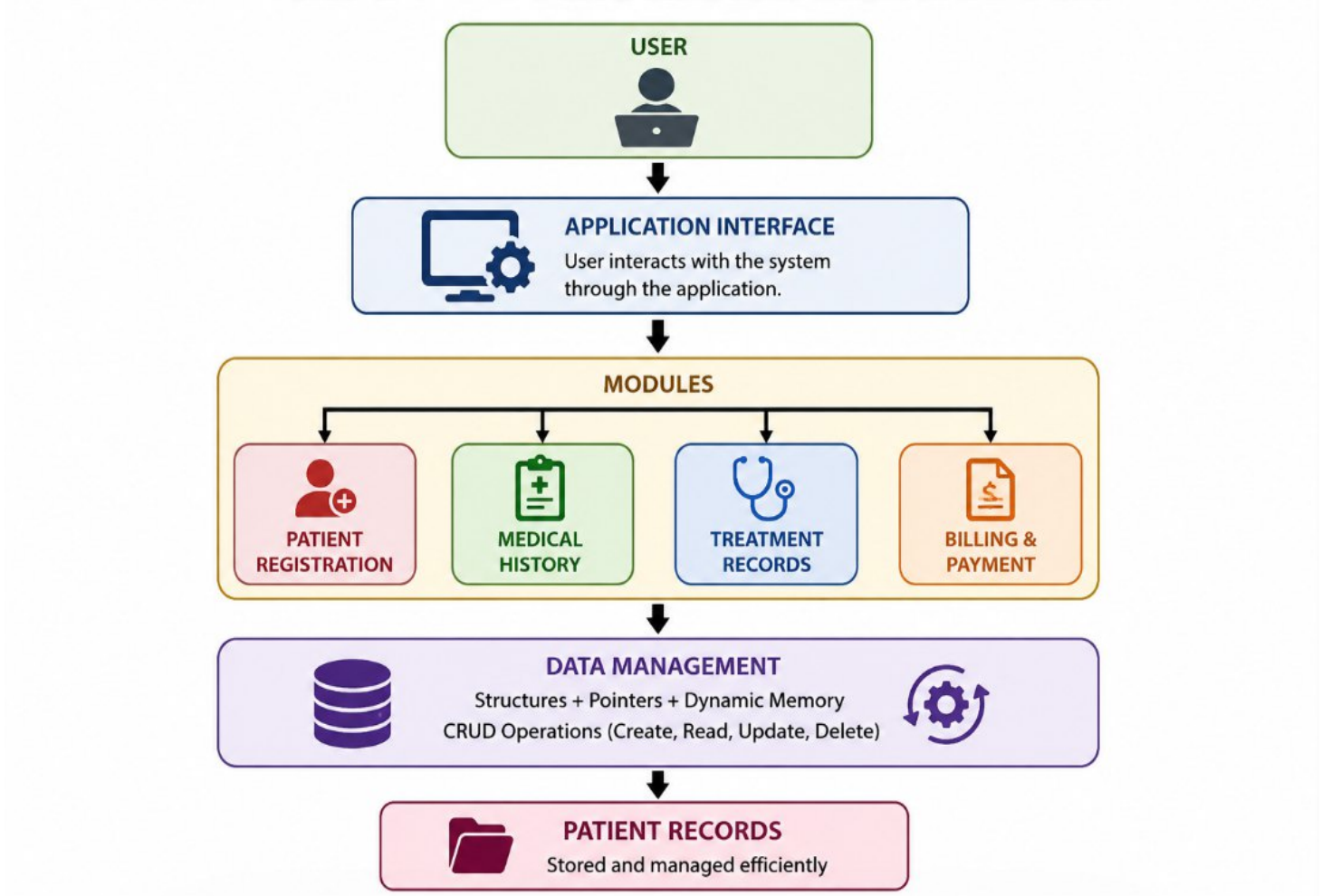
Treatment Records – Manage diagnosis, treatment, and prescriptions.

Billing & Payment – Calculate and manage patient bills.

CRUD Operations – Create, Read, Update, and Delete records.

Search & Access – Quickly retrieve required patient information.

Overall Architecture



Module 1 – Patient Registration

Add Patient – Enter and store new patient details.

View Patient – Display registered patient information.

Update Patient – Modify existing patient details.

Delete Patient – Remove unwanted patient records.

Uses **structures and pointers** for efficient record management.

Module 2 – Medical History

Stores the patient's **previous medical records** .

Maintains **past diseases and health conditions** .

Records **allergies and important medical information** .

Provides quick access to **patient medical history** .

Supports **viewing and updating** medical records**.**

Module 3 – Treatment Record

Records **doctor and patient details** .

Stores **diagnosis and health condition** .

Maintains **treatment and prescription details** .

Records **medicines and dosage information** .

Allows treatment records to be **viewed and updated**.

Module 4 – Billing & Payment

Generates **patient bills** based on treatment and services.

Records **payment details** and payment status.

Calculates the **total bill amount**.

Maintains **billing records** for each patient.

Supports **viewing and updating** payment information.

Data Structures Used

Structures – Store patient and hospital-related information.

Arrays – Manage multiple patient records efficiently.

Nested Structures – Organize medical history, treatment, and billing details.

Pointers – Access and modify structure data efficiently.

Dynamic Memory – Allocate and release memory when required.

Pointer-Based Access

Pointers store the memory address of patient records.

Use the **-> operator** to access structure members through pointers.

Enables **direct and efficient data access** .

Helps modify patient records without copying the entire structure.

Supports **dynamic memory management** using pointers.

Example:

```
patient_ptr->name;
patient_ptr->age;
```

Dynamic Memory Allocation

malloc() – Allocates a block of memory dynamically.

calloc() – Allocates and initializes memory to zero.

free() – Releases dynamically allocated memory.

Helps use **memory efficiently** according to requirements.

Supports flexible management of **patient records**.

Example:

```
patient = (Patient *)malloc(sizeof(Patient));

free(patient);
```

CRUD Operations

Create – Add new patient records to the system.

Read – View and retrieve existing patient information.

Update – Modify patient, treatment, or billing details.

Delete – Remove unwanted or outdated records.

Provides **efficient and organized record management** .

System Workflow & Testing

System Workflow

Start → Patient Registration → Medical History → Treatment → Billing & Payment → Record Update/View → End

Testing

Test **patient registration** with valid details.

Verify **medical history and treatment** records.

Check **billing and payment calculation** .

Test **CRUD operations** for correct results.

Verify **invalid inputs and error handling** .

Result: System functions correctly and manages patient records efficiently.

Outcomes & Future Enhancements

Outcomes

Improved **C programming and problem-solving skills** .

Efficient management of **patient records** .

Practical understanding of **structures and pointers** .

Better understanding of **dynamic memory allocation** .

Effective implementation of **CRUD operations** .

Future Enhancements

Add a **database** for permanent record storage.

Develop a **web/mobile interface** .

Add **user authentication and security** .

Enable **online appointment management** .

Conclusion

The system provides **efficient and organized patient record management** .

Structures and pointers enable effective data storage and access.

CRUD operations simplify record management.

Modules integrate **registration, medical history, treatment, and billing** .

The project improves **programming skills, memory management, and problem-solving** .

References

The C Programming Language – Brian W. Kernighan and Dennis M. Ritchie.

C Programming: A Modern Approach – K. N. King.

GeeksforGeeks – C Structures, Pointers and Dynamic Memory Allocation.

Programiz – C Programming Tutorials and References.

W3Schools – C Programming Concepts and Syntax.

THANK YOU

Thank you for your attention